

System Design Document

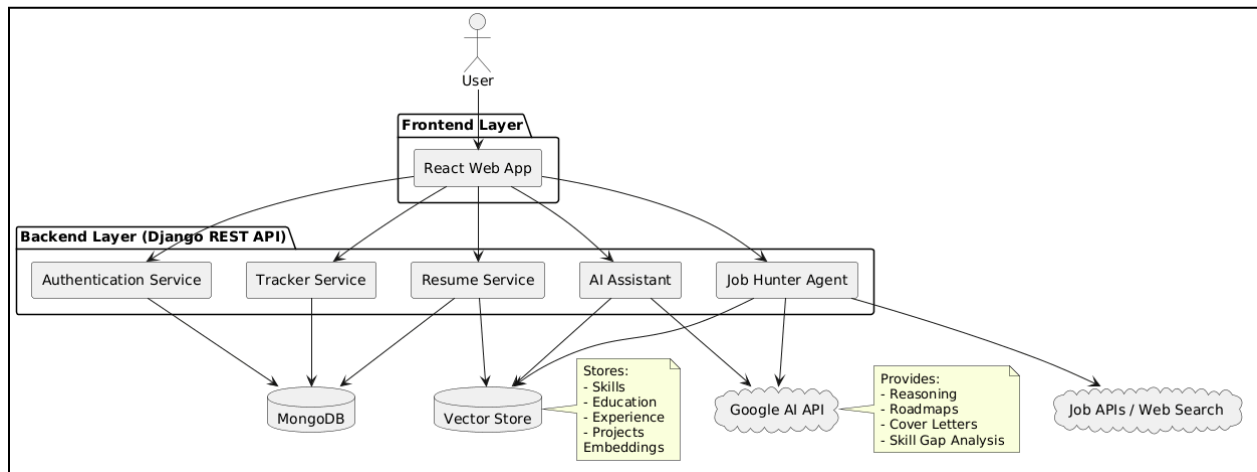
CareerPilot	1
System Architecture	1
Technology Stack	1
High-Level Data Flow	2
Component Design	3
Database Design	4
Scaling Strategy for 10,000 Users	5
Frontend	5
Backend	5
Database	5
Vector Search Scaling	5
Scaling Strategy for 10,000 Users - Diagram	6
Estimated Cost Analysis	7
Infrastructure Cost	7
Performance Optimization	8
Key Bottlenecks and Mitigation	9
Security Considerations	10

CareerPilot

CareerPilot is an AI-powered career co-pilot designed to assist job seekers throughout their career journey. The platform combines Resume Intelligence, Job Hunting, AI Career Assistance, and Productivity Tracking into a single ecosystem.

The system is built using React for the frontend, Django REST Framework for backend services, MongoDB for data storage, and Google AI for intelligent reasoning. A Retrieval-Augmented Generation (RAG) architecture serves as the foundation of the platform, ensuring that every recommendation and AI-generated response is grounded in the user's actual CV.

System Architecture

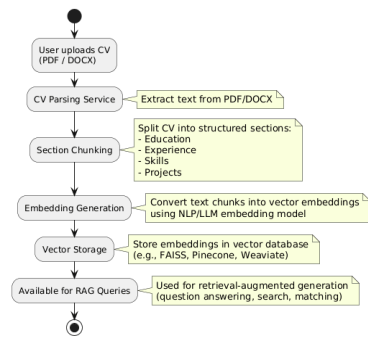


Technology Stack

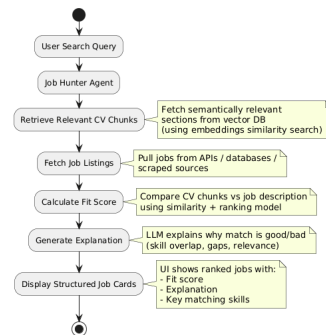
Layer	Technology
Frontend	React JS
Backend	Django
Database	MongoDB
ChatBot	Google AI

High-Level Data Flow

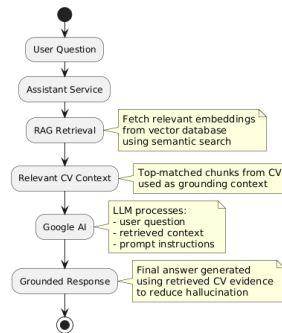
Resume Processing Flow



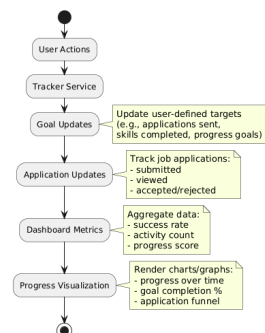
Job Matching Flow



AI Assistant Flow



Productivity Tracking Flow



Component Design

Resume Intelligence Module	Responsibilities: • PDF/DOCX Upload • Resume Parsing • Section Identification • Chunk Generation • Embedding Generation • Vector Storage	Output: • Searchable User Knowledge Base
Job Hunter Agent	Responsibilities: • Natural Language Search • External Job Source Integration • Candidate Matching • Fit Score Calculation • Recommendation Generation	Output: • Structured Job Cards • Match Explanations
AI Assistant	Responsibilities: • Career Guidance • Skill Gap Analysis • Roadmap Generation • Cover Letter Creation	Output: • Context-Aware Responses
Tracker Module	Responsibilities: • Goal Tracking • Task Management • Application Tracking • Progress Analytics • Reminder Generation	Output: • Dashboard Metrics • User Productivity Reports

Database Design

Users Collection

- User Information
- Authentication Data
- Profile Settings



Resume Chunks Collection

- User ID
- Chunk Content
- Chunk Type
- Embedding Vector



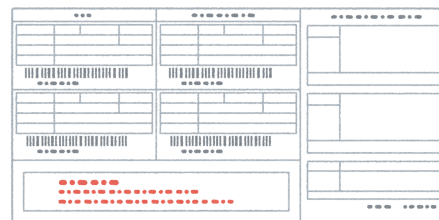
Jobs Collection

- Job Details
- Company Information
- Salary Information
- Deadlines



Applications Collection

- Job Applications
- Application Status
- Timeline History



Goals Collection

- Career Goals
- Completion Status
- Deadlines



Scaling Strategy for 10,000 Users

What if:

- 10,000 Registered Users
- 2,000 Daily Active Users
- Average 15 AI Requests Per User Per Day
- Average Resume Size: 2 MB



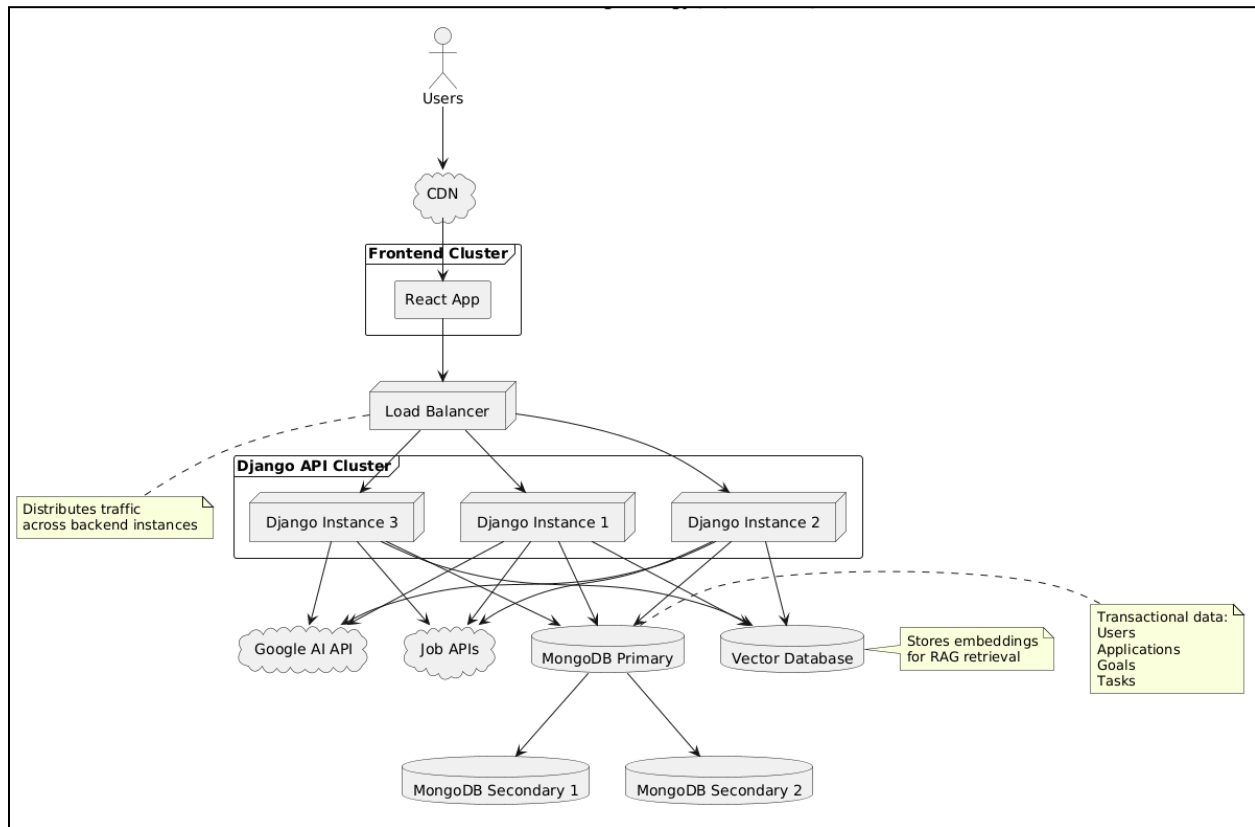
Solution



**Horizontal
scaling !!!**

Frontend	Backend	Database	Vector Search Scaling
Deploy React application using CDN distribution. Benefits: • Reduced latency • Global caching • Lower server load	Deploy multiple Django instances behind a Load Balancer. Example: → Django Instance 1 → Django Instance 2 → Django Instance 3 Benefits: • High Availability • Better Throughput • Fault Tolerance	Use MongoDB Replica Set: Primary Node ↓ Secondary Node ↓ Secondary Node Benefits: • Read Scalability • Automatic Failover • Data Redundancy	Store embeddings separately from transactional data. Benefits: • Faster similarity search • Independent scaling • Reduced query latency

Scaling Strategy for 10,000 Users - Diagram



Estimated Cost Analysis

Assuming per active user per month:

100 AI Requests	1 Resume Upload	50 Vector Searches
-----------------	-----------------	--------------------

Infrastructure Cost

Frontend Hosting	Backend Compute	Database Storage	Vector Storage	AI API Usage	Monitoring & Logging
\$0.05/User/Month	\$0.15/User/Month	\$0.05/User/Month	\$0.03/User/Month	\$0.40/User/Month	\$0.02/User/Month

Estimated Monthly Cost:

\$0.70 per active user

For 10,000 users:

$10,000 \times \$0.70$

$\approx \$7,000/\text{month}$

This cost can be reduced through:

- Response caching
- Batch embedding generation
- Cheaper embedding models
- Prompt optimization

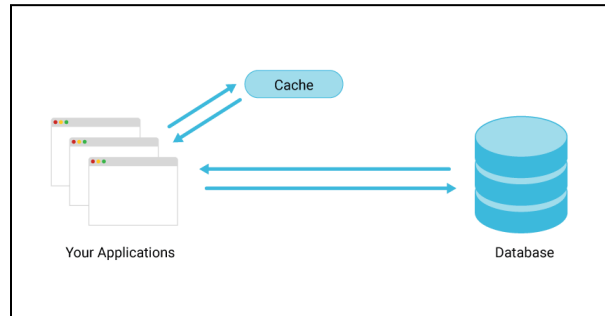
Performance Optimization

Caching

Cache:

- Frequently accessed job results
- Common AI responses
- Dashboard statistics

Expected improvement:
30–50% reduction in API calls.



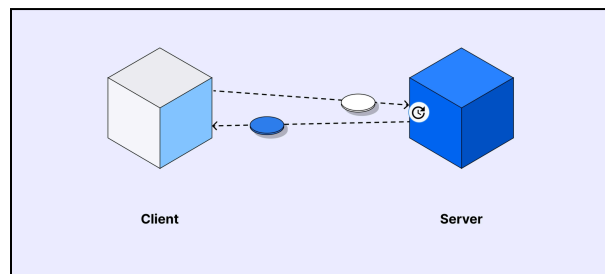
Asynchronous Processing

Move heavy operations into background workers:

- Resume parsing
- Embedding generation
- Job crawling

Benefits:

- Faster user experience
- Better scalability



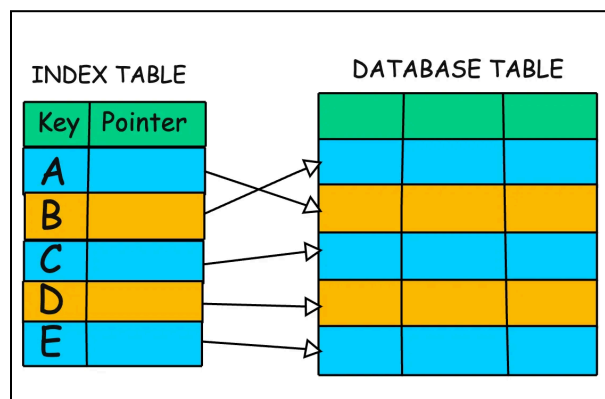
Database Indexing

Indexes:

- User ID
- Job or Goal Deadline
- Application Status

Benefits:

- Faster queries
- Reduced response time



Key Bottlenecks and Mitigation

Bottleneck	Issue	Mitigation
AI API Rate Limits	Large user volume may exceed API quotas.	• Request queue • Response caching • Fallback models
Vector Search Latency	Embedding retrieval becomes slower with growth.	• Dedicated vector database • Optimized indexing • Approximate nearest-neighbor search
Resume Processing	Large uploads increase processing time.	• Background processing • File size limits • Queue-based architecture
Job Aggregation	External job sources may throttle requests.	• Scheduled crawling • Cached job listings • Multiple data providers
Dashboard Analytics	Real-time analytics become expensive.	• Precomputed statistics • Scheduled aggregation jobs

Security Considerations

- JWT Authentication
- Password Hashing
- HTTPS Encryption
- Secure File Upload Validation
- Role-Based Access Control
- Environment Variable Protection
- Rate Limiting

CareerPilot utilizes a modular microservice-inspired architecture centered around a RAG-powered Resume Intelligence Engine. The architecture is capable of supporting 10,000 users through horizontal scaling, load balancing, asynchronous processing, and optimized vector search. The estimated operational cost is approximately \$0.70 per active user per month while maintaining responsive AI-powered career assistance and job recommendation services.